

Effective dating approach

Recording when every change to a student's record took effect — and making the HESA return read that history. Detailed design.

Executive summary

HESA Data Futures collects a student's **history within the year**: when their status changed, when they moved course, when each module started and ended. Our platform already stores effective dates for some changes (suspensions, extensions, FTE and programme changes, supervisors, funding), but **not** for student status or module enrolments, and several code paths overwrite values with no history at all. On top of that, the return mostly reads each student's **current** value and reports it for the whole year.

This document proposes:

- **One history pattern** for every fact that changes over time: rows with `valid_from / valid_to` and `recorded_at`, guarded by a database no-overlap constraint.
- **One write path** (a history service) that every change goes through, with the current value kept on the main row as a cache so screens are unaffected.
- **A return that reads periods** overlapping the reporting window instead of “latest”, mapping onto `StudentCourseSession`, `SessionStatus` and `ModuleInstance`.
- **Retrospective-change detection** against returns that have already been signed off.

Contents

1. Current state — what's dated, what isn't, and where history leaks
2. What HESA Data Futures expects
3. How established platforms do it, and the storage options
4. Core design — conventions, table shape, constraints
5. The history service — API and change algorithm
6. Entity-by-entity design
7. Reading history and building the return
8. Retrospective changes and signed-off returns
9. Edge cases and the rules for each
10. UI changes
11. Migration and backfill
12. Testing
13. Phasing, open decisions and sources

1. Current state

What the code stores today

Fact	Where	Dated?	Notes
Suspension, extension	student_lifecycle_event	Yes	start_date / end_date / actual_end_date; requested → approved
Mode / FTE change	student_lifecycle_event	Yes	previous_intensity_pct → intensity_pct with effective date
Programme change	student_lifecycle_event	Yes	previous_programme_id → new_programme_id, effective_date
Supervisors	supervisor_relationship	Yes	valid_from / valid_to (null = current), end_reason
Funding	funding_arrangement	Yes	valid_from / valid_to
Relationships, ICR placements, bench fees, affiliations	person / icr models	Yes	valid_from / valid_to
Student status	student.status	No	One enum column (prospective ... terminated). Overwritten; no dates, no history
Current programme / mode	student.programme_id, study_mode	Partly	Changed by approved events (dated) — but also directly editable (see below)
Module enrolment	module_enrolment	No	Only academic_year; status enrolled / completed / withdrawn / failed overwritten
Custom attribute values	student_custom_value	No	Overwritten

Where history leaks today

Even where the model could keep history, these write paths change values without it:

Write path	What it does	Risk
StudentService.update_student	Generic PATCH: setattr on any field in the payload	Can overwrite status, programme_id, study_mode with no event, no date and no approver. Withdrawal and termination have no dedicated flow, so today they happen here.
lifecycle.py (suspend / return / resume)	Sets student.status to suspended / active	The event is dated, but the status change itself isn't stored as history
completion/service.py	Sets student.status = completed	Completion date lives elsewhere; status history lost
Taught module updates	Generic update on module enrolment	Withdrawal from a module overwrites status; no withdrawal date

What the return reads today

Return input	Current behaviour	Problem
Programme	Split into periods via <code>programme_periods_for</code> when an academic year is given	Works — but derived on the fly per student (one events query per student)
Study intensity (FTE)	Latest approved intensity	A change in April is reported for the whole year
Funding	Only arrangements with <code>status = active</code> and <code>valid_to IS NULL</code>	Funding that ended during the year is missing from that year's return
Status	<code>student.status now</code>	No way to report the sequence of statuses the year contained
Modules	Not dated	No start/end dates to report

2. What HESA Data Futures expects

Data Futures is an in-year model: it collects the history of the student's engagement, not a year-end snapshot.

The three entities that carry most of the dates:

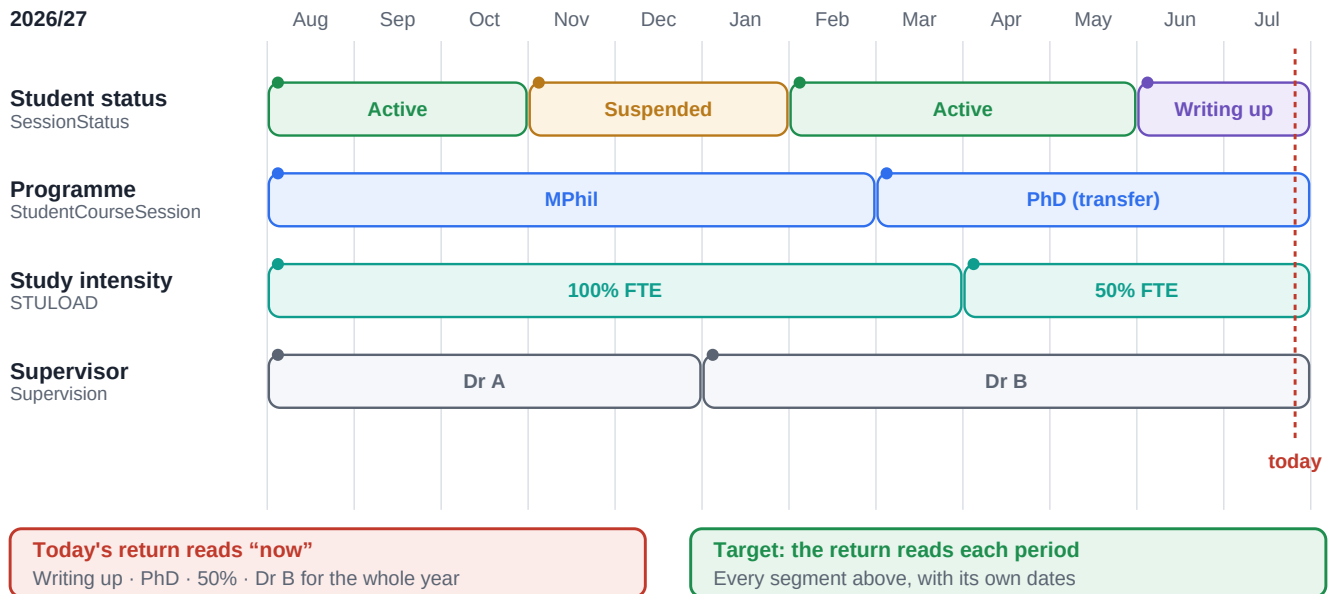
Entity	Dated fields	Rules that matter for us
StudentCourseSession	<code>SCSSTARTDATE</code> <code>SCSENDDATE</code>	A course change starts a new session. Status and module records sit inside a session.
SessionStatus	<code>STATUSCHANGEDTO</code> <code>STATUSVALIDFROM</code>	Child of the session. <code>STATUSVALIDFROM</code> is the date the student moved to that status and is part of the entity's key. An initial status is reported from the session start date. Used to monitor dormant and writing-up students, including when research students move to writing up for funding purposes.
ModuleInstance	<code>MODINSTSTARTDATE</code> <code>MODINSTENDDATE</code>	Per student, so it can differ from the cohort (e.g. a late joiner). If the student withdraws or suspends, the end date is that point. Must fall within the session's start and end dates. Returned in the reference period in which the session ends.

Implications:

- **Status needs real history**, including a writing-up state. Our `StudentStatus` enum has no writing-up value today.
- **Programme periods** become sessions; the existing programme-change events already give us the boundaries.
- **Module enrolments need their own dates**, clipped to the session and cut short by withdrawal or suspension.
- **Validation is date-based** (child dates inside parent dates), so dates must be consistent across tables, not just present.

One PGR student's year

Each bar is a period with its own effective date (the dot marks `valid_from`). HESA needs each period; today's return collapses them into the value at “today”.



3. How established platforms do it

- **PeopleSoft Campus Solutions.** Program and plan tables are keyed by EFFDT plus EFFSEQ (a sequence for several changes on the same day). Every change inserts a row; the current row is the latest one dated on or before today. Oracle builds its Data Futures return (SessionStatus, ModuleInstance ...) from these rows.
- **Tribal SITS** (widely used in the UK; general knowledge, not verified here). A course join record plus a per-year enrolment record; a course change creates a new course join — the same idea as a new StudentCourseSession.
- **Two kinds of time.** *Valid time* (when something was true in reality) vs *recorded time* (when the system learned it). A change dated last month but entered today can affect a period already submitted, so both are needed.
- **PostgreSQL.** Exclusion constraints (EXCLUDE USING gist) can forbid overlapping date ranges per student on any recent version. PostgreSQL 18 adds SQL-standard WITHOUT OVERLAPS keys and PERIOD foreign keys. Both need btree_gist.

Storage options considered

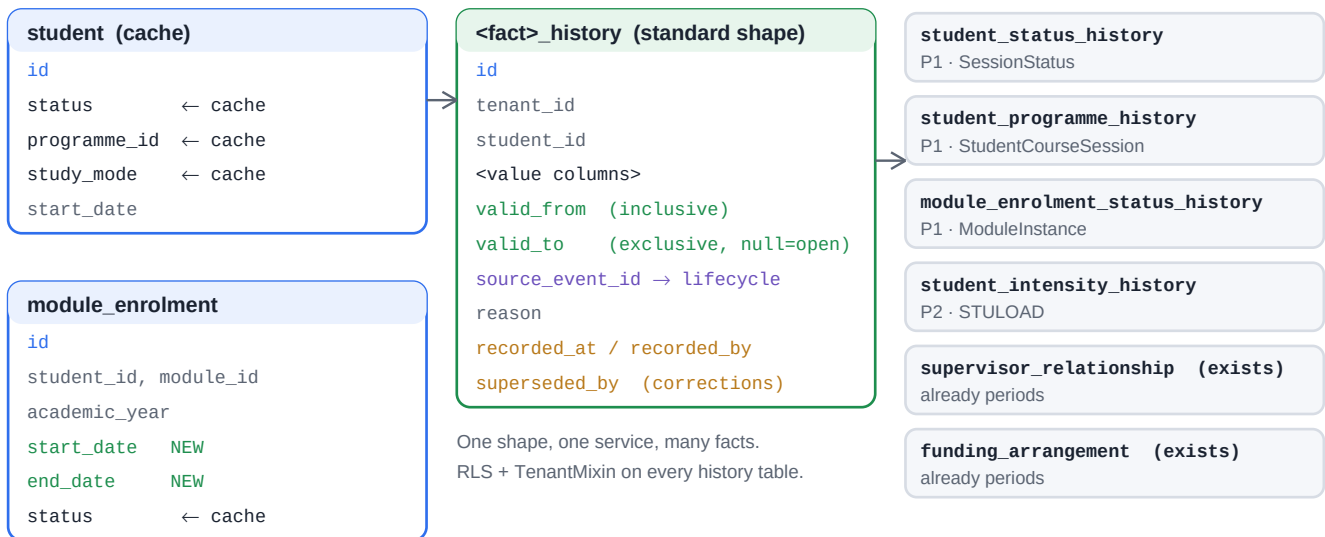
Option	How	For	Against	Verdict
A. Event log only	Keep events; derive state by replaying them	No new tables; events already exist	Every read replays events (the current <code>programme_periods_for</code> approach); hard to query, slow at scale; status changes without events are invisible	No
B. Row per change (EFFDT)	Insert a row per change with an effective date; current = latest ≤ today	Proven (PeopleSoft); simple inserts	End of a period is implicit (next row); “value during a window” needs window functions	Close
C. Periods (valid_from / valid_to)	Each row states its own start and end	Already our pattern (supervision, funding); overlap queries are trivial; DB can forbid overlaps	Changing a value must close the previous row — needs one write path	Chosen
D. Full bitemporal	Periods for valid time AND recorded time on every row	Complete “what did we know when”	Heavy; most questions don't need it	Partial: add <code>recorded_at</code> + <code>superseded_by</code>

4. Core design

Conventions

Rule	Decision	Why
Period shape	Half-open: <code>[valid_from, valid_to)</code> . <code>valid_to</code> is the first day it is no longer true; NULL = open-ended	Adjacent periods share a boundary date with no gaps or off-by-one errors; overlap test is one line
Dates	Calendar date (no time of day); UK institution calendar	HESA dates are dates; matches existing Date columns
HESA end dates	Converted at export: <code>HESA end = valid_to - 1 day</code>	HESA end dates are inclusive; our storage is exclusive
Contiguity	Status and programme: no gaps allowed. Supervision, funding, modules: gaps allowed	A student always has a status; they don't always have funding
Recorded time	<code>recorded_at</code> / <code>recorded_by_user_id</code> on every row	Needed to detect back-dated changes to signed-off periods
Corrections	Never hard-delete. A corrected row is kept with <code>superseded_by</code>	Audit: we can always show what the record said before
Tenancy	<code>TenantMixin</code> + RLS policy on every history table	Same isolation guarantee as the rest of the schema

The standard table shape



Example: student status history

Portable columns (the codebase keeps SQLite-compatible types for tests), with the no-overlap guarantee added as a PostgreSQL-only constraint in the migration:

```
CREATE TABLE student_status_history (
  id                uuid PRIMARY KEY,
  tenant_id         uuid REFERENCES tenant(id),
  student_id        uuid NOT NULL REFERENCES student(id) ON DELETE CASCADE,
  status            student_status NOT NULL,
  valid_from        date NOT NULL,           -- inclusive
  valid_to          date,                   -- exclusive; NULL = current
  source_event_id   uuid REFERENCES student_lifecycle_event(id),
  reason            text,
  recorded_at       timestampz NOT NULL DEFAULT now(),
  recorded_by_user_id uuid REFERENCES users(id),
  superseded_by     uuid REFERENCES student_status_history(id),
  CHECK (valid_to IS NULL OR valid_to > valid_from)
);

-- PostgreSQL only: no two LIVE periods may overlap for the same student.
CREATE EXTENSION IF NOT EXISTS btree_gist;
ALTER TABLE student_status_history ADD CONSTRAINT student_status_no_overlap
  EXCLUDE USING gist (
    student_id WITH =,
    daterange(valid_from, valid_to, '[') WITH &&
  ) WHERE (superseded_by IS NULL);

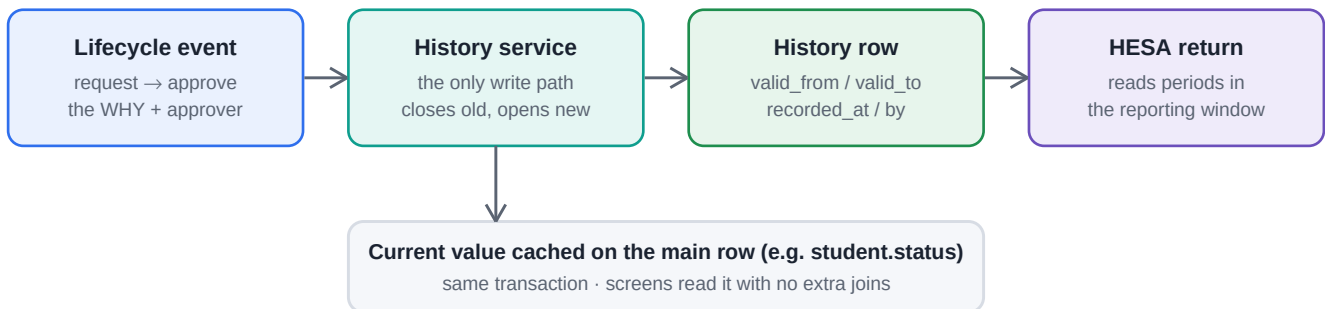
CREATE INDEX ix_ssh_student_from ON student_status_history (student_id, valid_from);
```

Why an exclusion constraint rather than WITHOUT OVERLAPS

PostgreSQL 18's `WITHOUT OVERLAPS` needs a single range column. Keeping the two date columns (our existing pattern) and using an exclusion constraint over `daterange(valid_from, valid_to)` gives the same guarantee, works on older PostgreSQL, allows the `WHERE superseded_by IS NULL` filter, and leaves the ORM model portable. The service still checks overlaps itself so SQLite tests behave the same.

5. The history service

One module owns every write to history tables. Nothing else may insert, update or close a history row, and nothing else may write the cached columns.



API

```

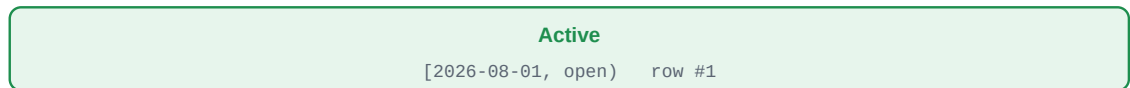
class HistoryService:
    async def change(self, fact: Fact, subject_id: UUID, value: dict, *,
                     effective_from: date, effective_to: date | None = None,
                     reason: str, user_id: UUID | None,
                     source_event_id: UUID | None = None) -> HistoryRow
        # reality changed on effective_from (optionally only until effective_to)

    async def correct(self, row_id: UUID, *, valid_from: date | None = None,
                     valid_to: date | None = None, value: dict | None = None,
                     reason: str, user_id: UUID | None) -> HistoryRow
        # the RECORD was wrong: supersede the row with a fixed copy

    async def value_at(self, fact: Fact, subject_id: UUID, on: date) -> HistoryRow | None
    async def periods(self, fact: Fact, subject_ids: list[UUID],
                      window: tuple[date, date]) -> dict[UUID, list[Period]]
    async def refresh_cache(self, fact: Fact, subject_id: UUID) -> None
  
```

Change algorithm

- 1 Validate: `effective_from` not before the student's start date; `effective_to` (if given) after it; permission to back-date if in the past.
- 2 Find the live row covering `effective_from`. If it starts on the same day, supersede it (same-day change — the EFFSEQ case) instead of creating a zero-length period.
- 3 Close it: set its `valid_to` to `effective_from`.
- 4 Insert the new row [`effective_from`, `effective_to` or old `valid_to`]. If `effective_to` is given (e.g. a suspension with an end), insert a third row that restores the previous value from `effective_to` to the old end.
- 5 If later rows already exist (a back-dated change), the new value runs only until the next recorded change — later history is never silently overwritten.
- 6 Refresh the cache: set the main-row column to the value covering today.
- 7 Run the retrospective check (section 8) and commit — all in one transaction.

Before

Approve: suspended from 2026-11-01 to 2027-01-31 (inclusive) → history service

After

Row #1 is closed (`valid_to` set); rows #2 and #3 are new. Half-open periods: `valid_to` is the first day it is NO LONGER true — no gaps, no overlaps. Cache: `student.status = value` covering today.

Change vs correction

	Change	Correction
Meaning	Reality changed on a date	The record was wrong
Example	Student suspended from 1 Nov	It was actually 8 Nov, entered wrongly
Effect	Closes a period, opens a new one	Supersedes the wrong row with a fixed copy; old row kept, marked <code>super_seded_by</code>
Permission	Normal approver	Elevated (registry) + mandatory reason
Return impact	New period appears	Period dates move; may affect a signed-off return

Future-dated changes

A change can be approved now with an effective date in the future (e.g. a mode change from next term). The history row is written immediately, but the cache only changes when the date arrives. A daily job on the existing scheduler calls `refresh_cache` for rows whose `valid_from` is today.

Closing the leaks

- `update_student` rejects `status`, `programme_id` and `study_mode` in a PATCH (409 with “use a lifecycle change”). Withdrawal and termination get proper lifecycle events with an effective date.
- Suspension, return, resume and completion call `HistoryService.change` instead of assigning `student.status`.
- Module withdrawal gets an explicit action with a date instead of a generic field update.

6. Entity-by-entity design**6.1 Student status → SessionStatus (P1)**

Status change	Triggered by	Effective date
registered	Enrolment	Registration / start date
active	Enrolment confirmed; return from suspension	Start date; actual return date
suspended	Approved suspension event	Suspension start; ends at its end date
writing up (NEW)	Progression milestone or registry action	Date agreed for writing-up
completed	Completion / award decision	Award date
withdrawn / terminated	NEW lifecycle event (today: generic PATCH)	Last date of engagement

Our statuses map to HESA's STATUSCHANGEDTO codes through a configurable code map, like the existing field mappings — the codes themselves come from the active spec pack, not hard-coded values.

6.2 Programme → StudentCourseSession (P1)

A `student_programme_history` table holds one period per programme. The initial row starts at the student's start date; each approved `programme_change` event closes it and opens the next at the event's `effective_date`. `programme_periods_for` keeps its signature but reads this table, so the return's existing per-period split keeps working and loses its per-student event replay.

6.3 Module enrolment → ModuleInstance (P1)

- `module_enrolment` gains `start_date` and `end_date`, defaulted from the module's term dates at enrolment and editable for late joiners.
- `module_enrolment_status_history` records enrolled → withdrawn / completed / failed with dates.
- Withdrawal sets `end_date` to the withdrawal date. When a student is suspended, open module enrolments are offered for closing at the suspension date (a proposal the registry confirms, not automatic — see open decisions).
- Validation: module dates must sit inside the programme period they belong to.

6.4 Study intensity → STULOAD (P2)

`student_intensity_history` is written by approved `intensity_change` and `mode_change` events; `study_mode` stays derived from it (100% = full-time). The return computes a day-weighted FTE for each period instead of the latest value:

```
FTE for period = sum(intensity_pct x days at that intensity) / days in period
```

```
Example (2026/27, 365 days): 100% for 243 days (Aug-Mar), 50% for 122 days (Apr-Jul)
(100 x 243 + 50 x 122) / 365 = 83.3%          today's return would report 50%
```

How dormant (suspended) days count towards STULOAD must be confirmed against the HESA coding manual before this is built.

6.5 Funding and supervision (P2)

Both already store periods. The change is on the read side: select arrangements and relationships that **overlap the reporting window** instead of only current ones (today funding that ended mid-year is dropped). Their write paths move behind the history service so `recorded_at` and overlap checks apply.

6.6 Other facts (P3)

Study location, fee status, and custom attribute values follow the same shape when a return needs them. For custom attributes, history is an opt-in flag per attribute so one-off fields don't pay the cost.

7. Reading history and building the return

Two query shapes cover every read:

```
-- Value on a date
SELECT * FROM student_status_history
WHERE student_id = :sid AND superseded_by IS NULL
      AND valid_from <= :on AND (valid_to IS NULL OR valid_to > :on);

-- Every period overlapping a window [:w_start, :w_end), for ALL students at once
SELECT * FROM student_status_history
WHERE superseded_by IS NULL
      AND valid_from < :w_end AND (valid_to IS NULL OR valid_to > :w_start)
ORDER BY student_id, valid_from;
```

- **Batch, don't loop.** The return runs one window query per fact for the whole cohort and groups in memory, replacing today's per-student event queries.
- **Clip to the window** in the service: `max(valid_from, w_start)` to `min(valid_to, w_end)`.
- **Screens don't change.** They keep reading the cached columns. Only the return, the student history tab and “as of” views read history tables.

Shape of the return record

Not every change splits the record. HESA's hierarchy decides:

Change	Effect on the return
Programme change	New StudentCourseSession (new record) — as today
Status change	Extra SessionStatus child inside the same session
Module enrolment	ModuleInstance child of the session
FTE change	Day-weighted value on the session
Supervisor / funding change	Values or children for the period, per the spec

The record contract `build_records` produces gains child lists (e.g. `statusHistory[], modules[]`) alongside today's flat fields, so existing mappings keep validating while Data Futures-shaped exports can read the children.

Performance

History tables grow by a few rows per student per year. With the `(student_id, valid_from)` index and one window query per fact, building the return costs roughly the same as today and removes the per-student event replay. Screens pay nothing because they read the cache.

8. Retrospective changes and signed-off returns

A change is **retrospective** for a return if its period overlaps that return's window **and** it was recorded after the return was signed off:

```
retrospective = row.valid_from < window_end
                and (row.valid_to is None or row.valid_to > window_start)
                and row.recorded_at > profile.signed_off_at
```

- **Before approval:** the event impact preview warns “This back-dates to 12 Mar; the 2026/27 return signed off on 1 Apr includes this student — it may need resubmitting.”

- **After approval:** the statutory page lists students with retrospective changes since sign-off, so registry can decide whether to resubmit.
- **Reproducibility:** because rows are never hard-deleted and carry `recorded_at`, the return can be rebuilt as it was at sign-off time.

9. Edge cases

Case	Rule
Two changes on the same day	The later one supersedes the earlier same-day row; no zero-length periods
Back-dated change before a later recorded change	Applies until the next recorded change; later history untouched; retrospective check runs
Future-dated change	Row written now; cache updated by the daily job on the effective date
Change before the student's start date	Rejected
Suspension overlapping a programme change	Allowed: facts are independent. The return splits sessions on programme and adds status children within each
Suspension with no end date	Open-ended suspended period; the return-from-suspension action closes it
Approved event later cancelled	A correction supersedes its rows; nothing hard-deleted; retrospective check runs
Withdrawn mid-module	Module <code>end_date</code> = withdrawal date; status child → withdrawn
Leap years / partial years	FTE weighting counts real days in the period
Gaps	Forbidden for status and programme (service fills with the previous value); allowed for supervision, funding, modules
Student deleted	ON DELETE CASCADE removes history with the student, as for events today

10. UI changes

- **Effective date on every change form** (status, programme, mode, module withdrawal). Defaults to today. Back-dating needs a permission and a reason; future dates allowed.
- **History tab on the student record:** the timeline shown in section 2, built from the history tables, with who recorded each change and when.
- **Retrospective warnings** in the approval dialog and on the statutory page.
- **Optional “as of” date** on the student record to view it as it was on a past date.
- **Generic edit forms** stop offering status / programme / mode fields and point to the lifecycle actions instead.

11. Migration and backfill

Additive Alembic migrations, then a one-off backfill that reconstructs as much real history as the events allow. Backfilled rows are marked (`reason = 'backfill'`) so they're distinguishable from live changes.

Table	Backfill source
<code>student_status_history</code>	Rebuilt from approved suspension events (active → suspended → active) plus the current status; anything else is one period from the start date
<code>student_programme_history</code>	Replay of approved programme-change events (the same logic <code>programme_periods_for</code> uses today)
<code>student_intensity_history</code>	Replay of approved intensity / mode events; before the first, the mode at start (100% or the part-time default)
<code>module_enrolment_dates</code>	From the module's term, otherwise the academic-year window (1 Aug – 31 Jul)
<code>module_enrolment_status_history</code>	One period with the current status

After backfill, a check asserts that every cached column equals `value_at(today)` for every student. The migration fails loudly if they disagree.

12. Testing

Test	What it proves
Invariants (property-based)	After any random sequence of changes: no overlaps, no gaps where forbidden, cache = value at today
Change algorithm	Same-day supersede, back-dated insert, future-dated change, suspension with and without an end
Constraint	PostgreSQL rejects overlapping live rows even if the service is bypassed
Leak guards	PATCH with <code>status / programme_id</code> returns 409
Return golden file	The section 2 example student produces the expected sessions, status children and 83.3% FTE
Retrospective check	A back-dated change after sign-off is flagged; one before isn't
Tenancy	History rows are invisible across tenants (existing RLS test pattern)
Backfill	Reconstructed periods match existing events on the seeded data

13. Phasing

Phase	Scope	Outcome	Size
1	History service, student_status_history, withdrawal/termination events, close the PATCH leak, backfill	Every status change is dated and approved; SessionStatus data exists	M
2	student_programme_history + student_intensity_history; return reads them in batch; day-weighted FTE	Correct sessions and STULOAD; no per-student replay	M
3	Module enrolment dates + status history; withdrawal action; date validation	ModuleInstance data exists and validates	M
4	Return reads funding / supervision by overlap; record child lists	In-year funding and supervision reported correctly	S
5	Retrospective detection + UI history tab + effective-date inputs	Registry sees impact on signed-off returns	M
6	P3 facts; custom-attribute history opt-in	Remaining return fields	S–M

Open decisions

Decision	Options	Recommendation
Writing-up status	Add to StudentStatus / derive from a milestone	Add it; set by a progression milestone or registry action
Modules on suspension	Close automatically / propose to registry	Propose; registry confirms (avoids closing modules the student will resume)
Back-dating limit	Unlimited / within the open return year / elevated permission beyond	Free within the open year; elevated permission + reason before it
STULOAD and dormant days	Include / exclude suspended days	Confirm with the HESA coding manual before phase 2
Custom attribute history	Always / opt-in	Opt-in per attribute

Sources

- [HESA — Session status \(SessionStatus\)](#)
- [HESA — SessionStatus.STATUSVALIDFROM](#)
- [HESA coding manual — STATUSVALIDFROM](#)
- [HESA — Module instance \(ModuleInstance\)](#)
- [HESA coding manual — MODINSTSTARTDATE](#)
- [HESA coding manual — MODINSTENDDATE](#)
- [Oracle — Data Futures Return, Session Status entity](#)
- [Oracle — Data Futures Return, Module Instance entity](#)
- [PeopleSoft ACAD_PLAN table \(EFFDT / EFFSEQ\)](#)
- [PostgreSQL 18 temporal constraints \(Neon\)](#)
- [Temporal constraints in PostgreSQL 18 \(Better Stack\)](#)